Follow along
scan for the slides

Why Software Engineering Best Practices Fail in Data Engineering

Constance Martineau

Constance Martineau

Staff Product Manager · Astronomer

The data in this talk comes from Astro, the managed Airflow platform I work on.
I used to cosplay as a data engineer.

github.com/cmarteepants
linkedin.com/in/constance-martineau

Pipeline fails. The instinct: blame the last deploy.

POSSIBLE CAUSES

- Schema drift
- Late arrival
- Env change

→ **Last deploy**

In application development, that instinct is right.

Code controls behavior.

Data engineering inherited the playbook wholesale.

CI/CD for DAGs | Code review | Tests | Rollbacks

All built around the deploy boundary.

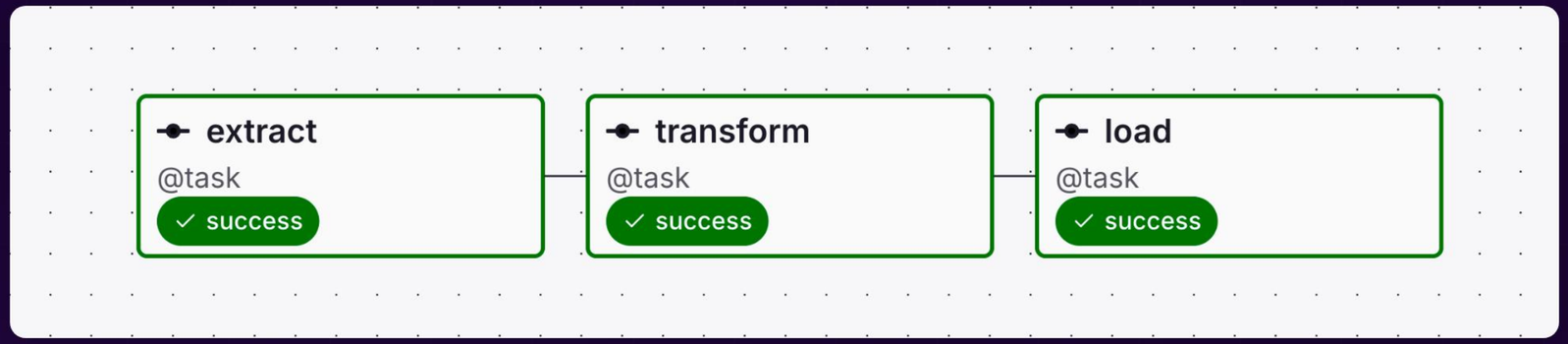
What if the instinct is right, for the wrong reasons?

I looked at 400 million production pipeline runs to find out.

400M

PRODUCTION PIPELINE RUNS

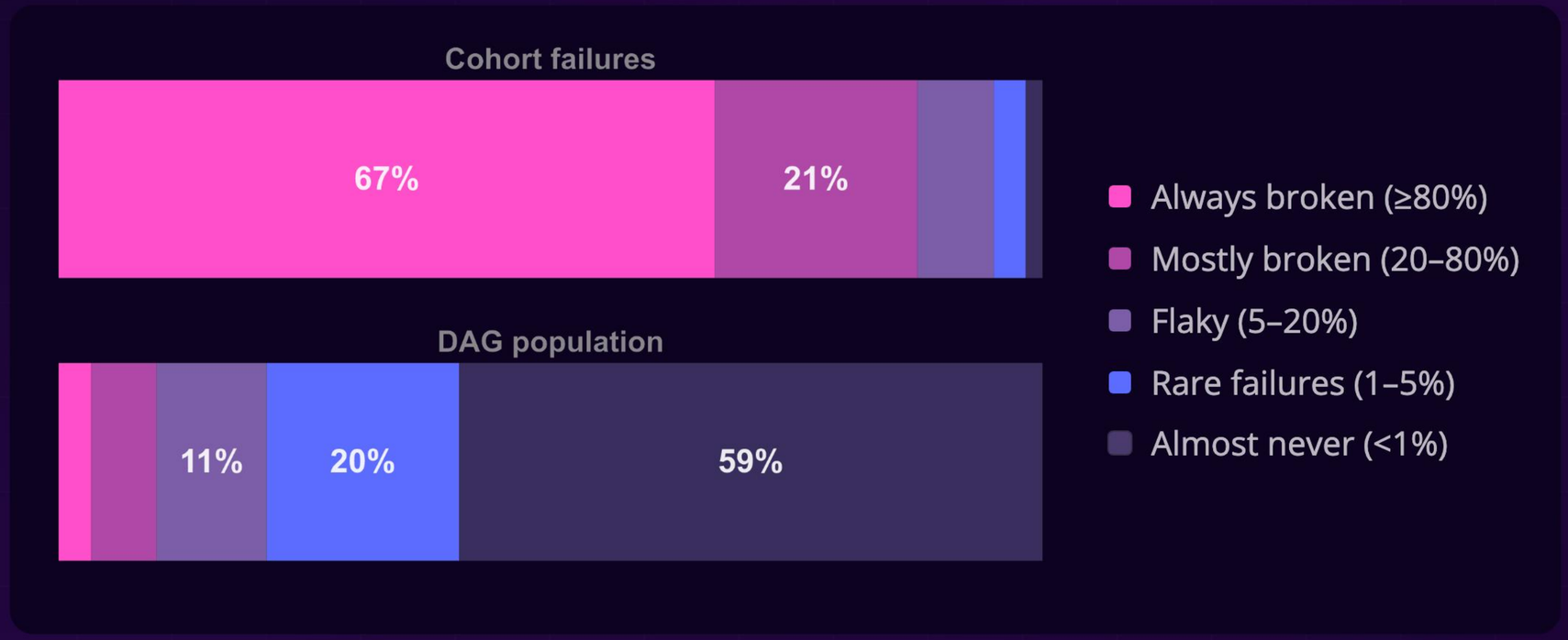
Astronomer managed platform | 2025 calendar year | paying customers only



A DAG: A WORKFLOW IN AIRFLOW

3% of DAGs cause 67% of failures.
60% almost never fail.

Cohort average: **7%** · Strip the chronic failures, the rest runs at **2%**.



Where the failures live

~½

OF FAILURES

concentrated on one runtime version

3–5×

FAIL RATE

long-running vs. fast workloads

97%

FAILURE RATE

thousands of always-broken DAGs

None of these trace back to a deploy.

THE TEST

BEFORE

24 hours

DEPLOY

AFTER

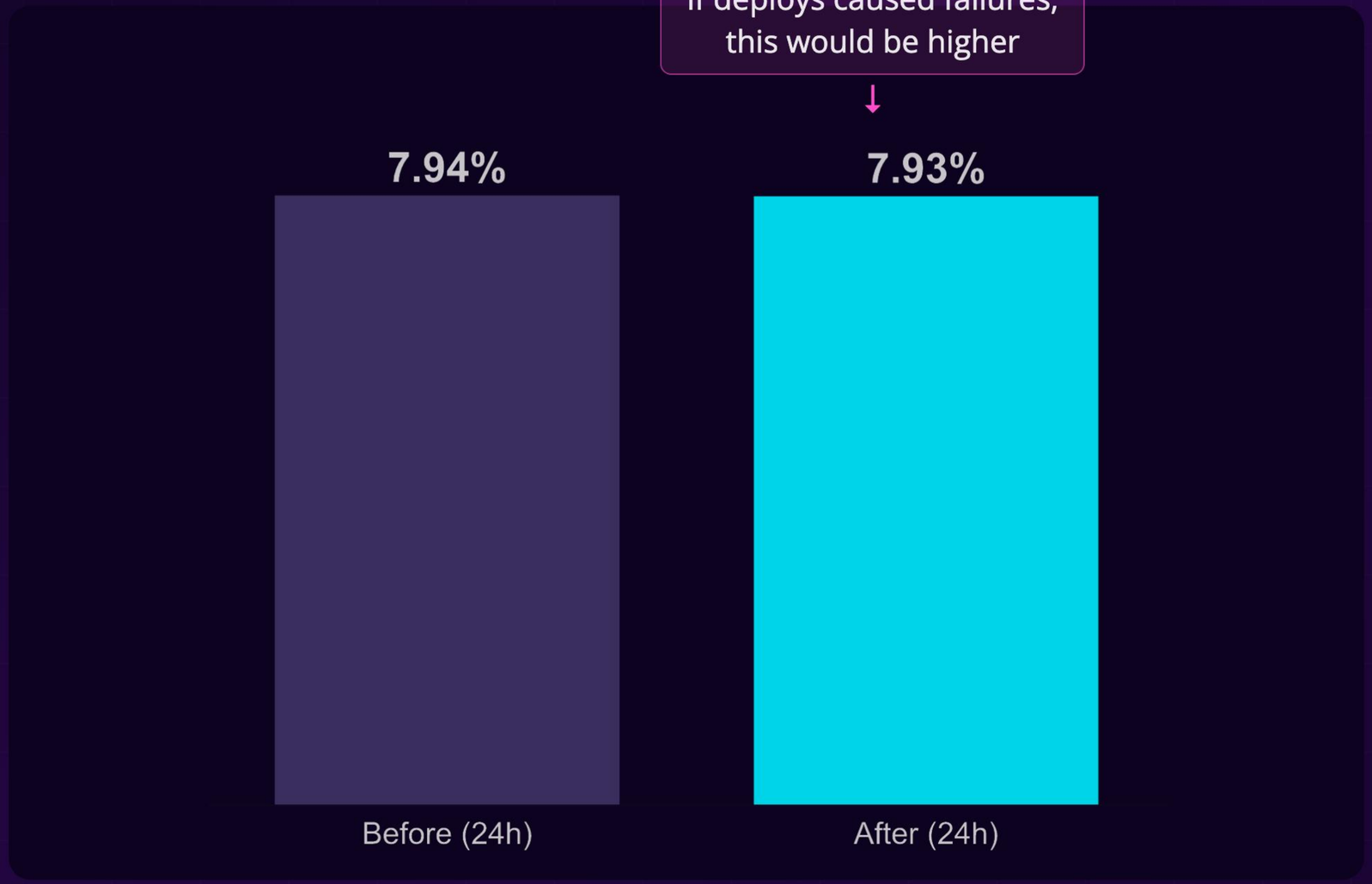
24 hours

same environment

same DAGs

adjacent windows

The failure rates are essentially identical.



The deploy is an attention anchor, not a causal one.

EARLIER TODAY

08:15 • dag_pipeline_sales · failed

09:47 • dag_reporting_daily · failed

11:03 • dag_pipeline_sales · failed

▲ AIRFLOW 2.9 → 2.10 · 14:32

14:33 • **dag_pipeline_sales · failed**

"it worked fine before 2.10!"

So if the deploy didn't cause it, what did?

A CONVERSATION WE KEPT HAVING

DAG 1: vendor API
`requests==2.25`
auth library pins it

⚡

DAG 2: ML model
`requests>=2.28`
async support required

> *"Just pick a version and adjust the DAG code, or containerize the DAGs."*
>
> senior platform engineer (someone I respect)


IT'S NOT A CODE PROBLEM.
BUT YOU CAN SOLVE IT WITH CODE... RIGHT?
...RIGHT?

THE MENTAL MODEL THAT DOESN'T TRANSFER

SOFTWARE ENGINEERING

Code

↓ controls

Inputs & behavior

↓ produces

Outputs

Code drives outputs

VS

DATA ENGINEERING

Upstream data

↓ shapes

Code structure & deps

↓ processes

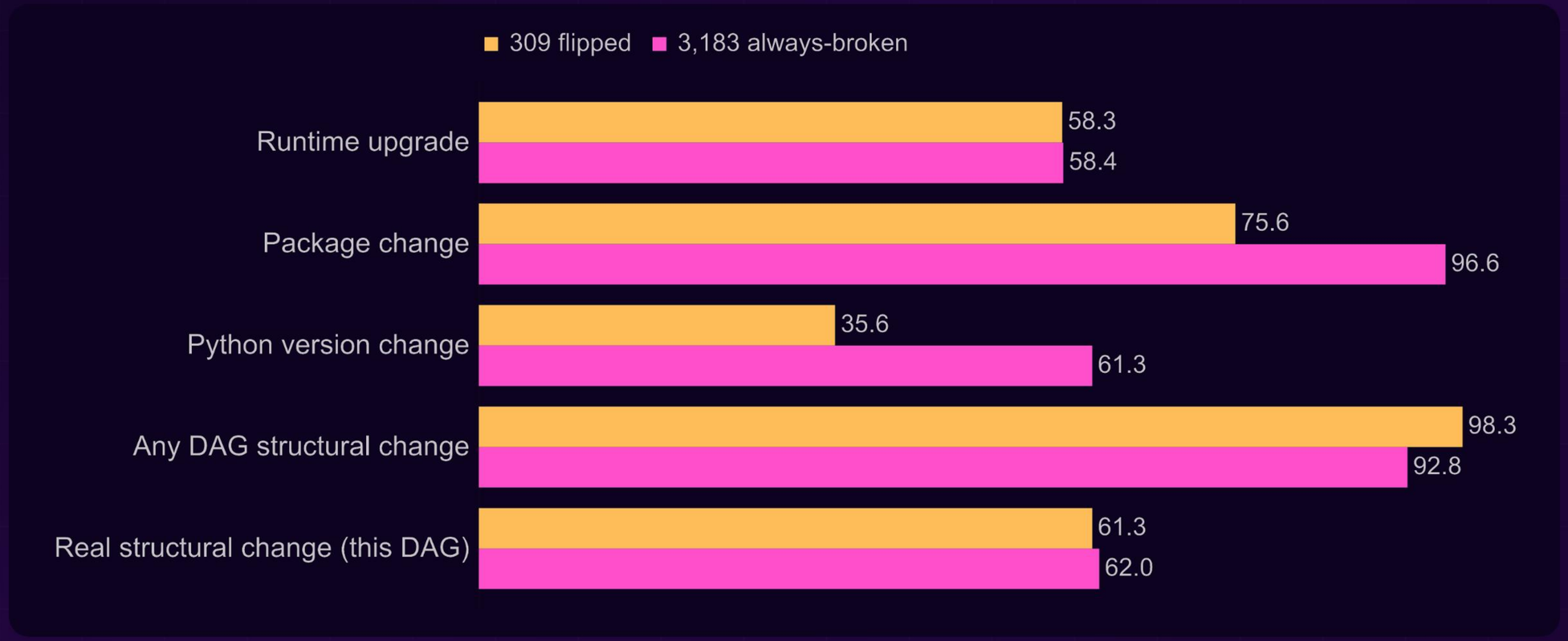
Outputs

Data shapes the code

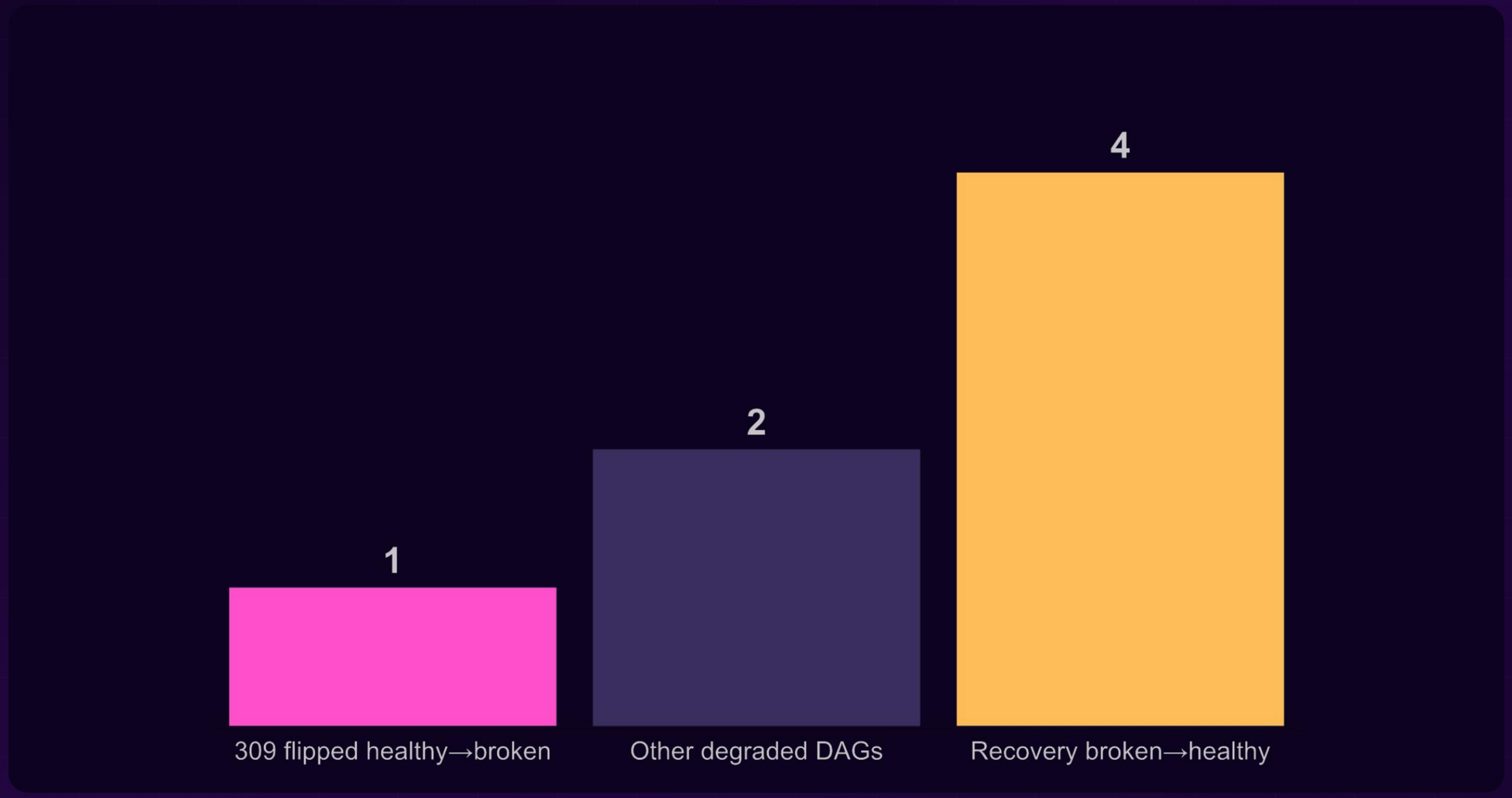
Your tests pass. Your CI/CD is green. And then upstream drops a column.

●	pytest	all 247 tests passed
●	CI / CD	build #1847 passed
●	pipeline run	KeyError: 'user_type'

No more code changes than DAGs that were broken from day one.



The recovery cohort had 4× the code changes.



The absence of a code-change signal is the signal.

Likely culprit: upstream data variance.

- schema drift
- late arrivals
- input shifts

- ~~runtime upgrades~~
- ~~package changes~~
- ~~business logic~~
- ~~Python versions~~

DEV

15%

PROD

7%

Prod is what survives.

Deploy practices cover layer 1 of a five-layer problem.

The five layers of pipeline reliability

1 **Code correctness**
Unit tests, lint, code review. CI/CD lives here. Real, just not the whole story.

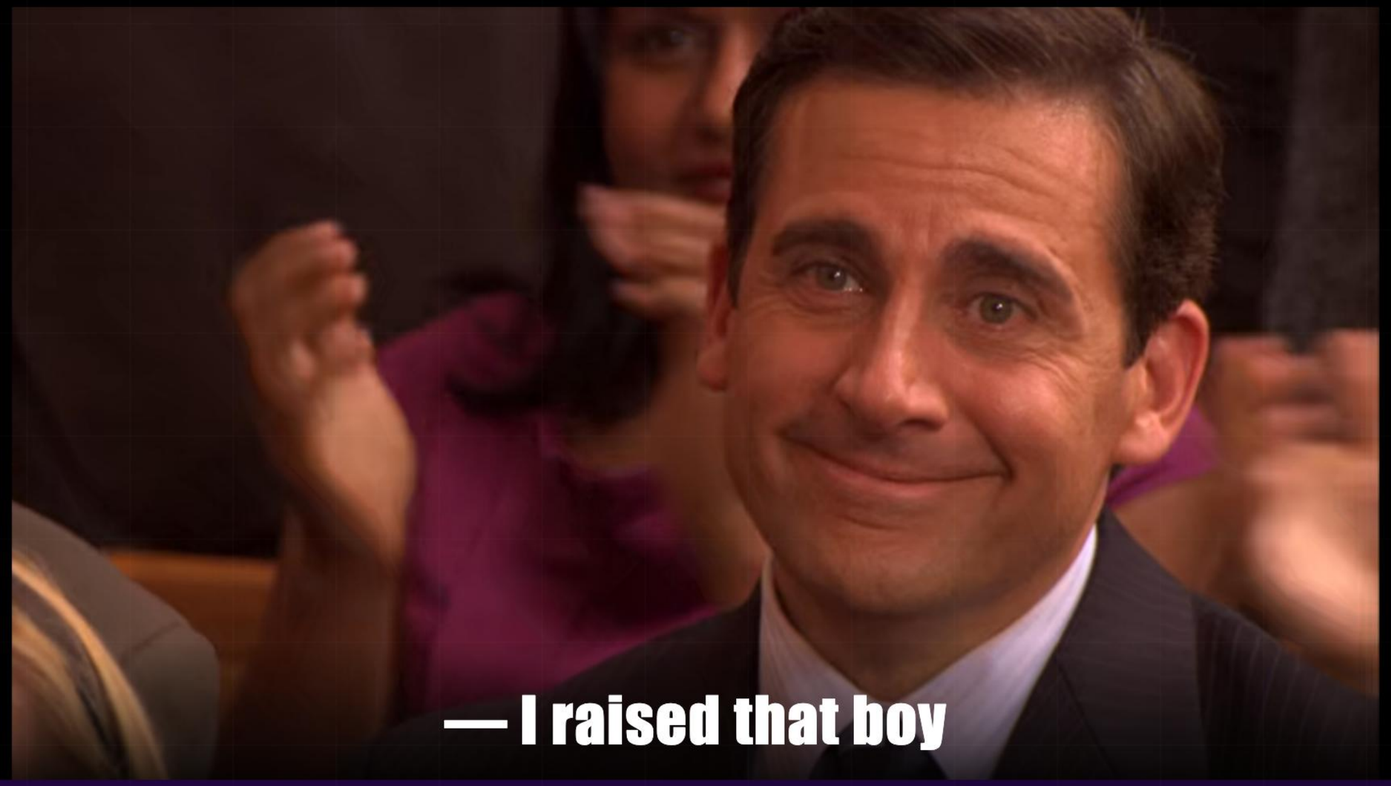
2 **Data correctness**
Null checks, uniqueness, schema validation. Code is correct; data may not match.

3 **Time correctness**
Freshness, lateness, completeness by cutoff. A "correct" DAG can still process stale data.

4 **State transition safety**
Write-audit-publish, staged tables, publish gates. The pipeline ran. But is the write safe to read?

5 **Recovery and degradation**
Dead-letter paths, stale-but-labeled data, idempotent backfills. Trustable data, even when degraded.

We borrowed the CI/CD playbook and declared victory.



What actually works

01 **Contract-test your inputs, not just your code**

Define what you expect from upstream. Alert when it drifts. Your test suite assumes stable inputs. Instrument the inputs themselves.

02 **Treat time as a first-class concern**

SLA misses and late-arriving data are failures. Instrument them as such. "Ran on schedule" and "processed fresh data" are different things.

03 **Design for the input surprise**

Graceful degradation is the reliability strategy for systems whose primary failure surface is outside your control.

04 **Know your always-broken DAGs**

3% are generating 67% of your failures. Triage them differently. Don't let them inflate your reliability metric or your on-call burden.

Time and data state are first-class concerns.

Design for the input you'll get, not just the input you expected.

Software engineering best practices didn't fail in data engineering.

They got applied to layer 1 of a five-layer problem and mistaken for the whole thing.

1 Code | 2 Data | 3 Time | 4 State | 5 Recovery

Build for the other four.

Thank you.

Questions?

LINKEDIN.COM/IN/CONSTANCE-MARTINEAU